

# SCAR: Constant-Size State-Carrying Memory with Attentive Recall

A controlled study of supervision density, length extrapolation, and memory ablations

Arjun K. Shah

September 2026

## Abstract

Can a recurrent model retain useful information over long sequences without growing a token-level memory? We introduce SCAR (State-Carrier with Attentive Recall), a compact architecture that combines a GRU state-carrier with a constant-size bank of 16 learned-decay exponential memory slots and one attentive read head. We evaluate SCAR against eight matched-size baselines and ablations on mod-2 parity, a random five-state automaton, and delayed first-token recall under dense and sparse supervision. The release contains 135 runs (nine models, five conditions, three seeds) and evaluates lengths up to 256 operations for parity/five and 512 for recall. SCAR matches the strong baselines at the trained lengths, but the ablation pattern is more informative than the headline accuracy: removing memory or its read path does not hurt the short training distribution, yet both ablations degrade on the longest recall probe. The dedicated CPU benchmark measures SCAR at roughly three times GRU training cost and below the RLT-lite baseline, so the result is not a claim that SCAR is universally cheaper. It is a controlled, synthetic finding that constant-size memory can matter for length extrapolation even when state carriage alone is sufficient for in-distribution fitting. We also document and test three corrected harness bugs before reporting the results. At the trained length, the best parity-dense result is Elman RNN ( $100.0 \pm 0.0\%$ ), with the same rounded mean as LSTM, GRU, TokenMerge (RLT abl.), RLT, SCAR, SCAR-carrier (no memory), SCAR-no-recall. On the longest delayed-recall evaluation, SCAR reaches  $100.0 \pm 0.0\%$ , compared with  $56.6 \pm 18.5\%$  for the carrier-only ablation and  $30.5 \pm 32.4\%$  when the memory is written but not read.

## 1 Introduction

Sequence models differ in how they carry information from the past. A recurrent network compresses history into a fixed-size state, while a causal Transformer exposes the whole prefix through an attention computation and an  $O(T)$  key-value cache [1, 2, 3, 4]. Memory-augmented models make the trade-off explicit by adding a writable storage mechanism and a learned read operation [5].

This paper asks a narrower question than “which architecture is best?” A recent Recurrent Looped Transformer (RLT) design carries decoder state while also reading a growing encoder-derived memory [10]. If a small RLT-like model succeeds on state-tracking tasks, which ingredient deserves credit: the recurrent carrier, the growing memory, or the looped read path? The answer matters because these mechanisms have different inference-state costs. SCAR is a subtractive test: it keeps a gated recurrent carrier and a cheap read path, but replaces the growing memory with a fixed bank of multi-timescale exponential summaries.

Our contributions are:

1. a precisely specified constant-size state-carrier architecture with an  $O(1)$ -per-token update and a fixed  $O(kd)$  memory state;
2. a matched-budget, three-seed sweep that crosses architecture with supervision density and evaluates beyond the training length;
3. parameter-matched memory and read-path ablations that separate short training performance from long-delay extrapolation; and

4. a reproducible methodology audit: the earlier release had a collapsed decay initialization, a BOS train/evaluation mismatch, and an incorrect description of the Transformer baseline. All reported numbers below come from the corrected harness.

## 1.1 Hypotheses

We began with two falsifiable hypotheses. First, SCAR should retain the useful state-carriage behavior of RLT while avoiding its growing memory and looped decoder. Second, its fixed read path might approach the GRU’s measured cost while remaining well below RLT-lite. The results support the first hypothesis only for long-horizon extrapolation and falsify the second in its strong form: SCAR is  $3.12\times$  the GRU training-step cost, although it is  $0.40\times$  the RLT-lite cost.

## 2 Related work

Elman-style recurrent networks, LSTMs, and GRUs provide progressively more structured mechanisms for carrying state through a sequence [1, 2, 3]. Transformers replace recurrence with self-attention and expose all earlier positions during a causal forward pass [4]. External-memory networks add a separately addressable storage and read mechanism [5]; structured state-space models offer another route to long-range sequence processing with fixed-size state [6]. Segment-level recurrence and compressed memories offer related ways to extend context beyond a fixed window [7, 8]. Associative recall and its multi-query formulation provide a particularly relevant diagnostic for whether an efficient sequence model can retrieve several previously presented items [9]. SCAR is intentionally small and synthetic: it is an ablation instrument for the state-carriage versus growing-memory question, not a claim to improve language modeling or to replace these broader model families.

The RLT baseline here is a faithful small-scale implementation of the publicly described recurrent-looped pattern: a causal encoder, a recurrent decoder state, global cross-attention to encoder outputs, and a sliding-window decoder cache. It should be read as an RLT-lite baseline, not as a reproduction of every large-scale result in the original report [10].

## 3 SCAR

Let  $e_t$  be the embedding of token  $x_t$  and  $h_t$  the GRU carrier state:

$$h_t = \text{GRU}(e_t, h_{t-1}), \quad (1)$$

$$u_t = \sigma(W_g[e_t; h_t]) \odot h_t, \quad (2)$$

$$m_{t,i} = \lambda_i m_{t-1,i} + (1 - \lambda_i) u_t, \quad i = 1, \dots, k. \quad (3)$$

Each slot is an exponential moving average. We initialize the learned decay parameters in logit space so that  $\lambda_i$  spans  $[0.90, 0.999]$ ; the corresponding half-lives span approximately 6.6 to 693 tokens. The memory therefore has a fixed  $k = 16$  slots and never grows with sequence length.

The read head uses a single query from the carrier:

$$q_t = W_q \text{LN}(h_t), \quad (4)$$

$$r_t = h_t + W_o \text{softmax}\left(\frac{q_t(W_k M_t)^\top}{\sqrt{r}}\right) W_v M_t, \quad (5)$$

$$y_t = W_{\text{out}} \text{LN}(r_t), \quad (6)$$

where  $M_t$  stacks the  $k$  slots and  $r = 40$  is the attention width. Each token performs a fixed amount of work independent of  $t$  and retains only the carrier and  $k$  memory slots. This gives  $\mathcal{O}(T)$  sequence work and  $\mathcal{O}(kd)$  recurrent state, in contrast to the growing  $\mathcal{O}(T)$  storage used by token-level KV memory. The constant-size memory does not make SCAR free: the dedicated timing study below measures its actual CPU cost.

## 4 Experimental design

### 4.1 Tasks and supervision

The parity task emits binary symbols and asks for their parity. The five-state task samples symbols from a fixed random transition automaton and asks for the final state. Their chance accuracies are 50% and 20%. The recall task samples the first symbol from an eight-symbol vocabulary and fills the remaining positions with independent distractors; the answer is the first symbol, giving a 12.5% chance floor. Recall is sparse-only because a repeated answer at every position would change the task into a different supervision problem.

Dense conditions supervise the state after every input symbol. Sparse conditions supervise only the final answer. All sequences are prefixed with a dedicated BOS token during both training and evaluation; sparse accuracy is measured at the final position. The main parity-sparse condition is fixed at 32 operations. A separate 4-to-8-to-16-to-32 curriculum condition was run during development, is archived under “results\_curriculum/“, and is excluded from the five-condition release tables so supervision density is not conflated with curriculum.

### 4.2 Models and protocol

The sweep contains Elman RNN, LSTM, GRU, a full causal Transformer, TokenMerge, RLT-lite, SCAR, SCAR-carrier (memory removed), and SCAR-no-recall (memory written but never read). Widths are selected to keep models within roughly 75–92K parameters; the two SCAR ablations widen the carrier to compensate for removed modules. All models use AdamW with learning rate 3e-3, weight decay 0.01, a 200-step warmup, cosine decay, 2,500 steps, and batch size 64. The release contains 135 matched runs: nine models, five task/supervision configurations, and three seeds (0–2). All models use AdamW (learning rate 3e-3, weight decay 0.01), a 200-step warmup, cosine decay, 2,500 optimizer steps, and batch size 64. Parity and five-state runs train on 32 operations; delayed recall trains on 64. All five release conditions use fixed training lengths; the separate curriculum condition is archived outside the release tables. Evaluation uses 2,048 fresh programs per length at 16–256 operations for parity/five and 64–512 for recall. Dense supervision scores the target after every input token; sparse supervision scores one answer after the chain. Training and evaluation both prepend one BOS token and score the final position for sparse tasks.

Each evaluation point uses 2,048 fresh programs. The release reports the mean, sample standard deviation, minimum, maximum, and per-seed values for every model/configuration/length cell. The headline table is explicitly at the trained length; the second table is explicitly extrapolation.

## 5 Methodology audit

Before the corrected sweep, we found three errors in the experiment harness:

1. **Collapsed decay initialization.** The old code stored  $\log \lambda$  but then applied a sigmoid, giving effective decays near 0.5 for every slot. The corrected code stores  $\text{logit}(\lambda)$ , restoring the intended multi-timescale range.
2. **Train/evaluation protocol mismatch.** Training prepended BOS but evaluation did not. The corrected evaluator applies the same prefix and scores the same final position; a protocol-sensitive regression test catches a future regression.
3. **Incorrect baseline description.** The Transformer implementation uses full causal attention, not a sliding window. The model code is unchanged; the paper, site, and tests now describe it accurately.

The earlier artifacts remain quarantined in “results\_v1/“ and “data/summary\_v1.json“. They are not aggregated into the release reported here.

Table 1: Accuracy (%) at the trained length (parity/five: 32 operations; recall: 64 operations), mean $\pm$ std over seeds. Chance is 50% for parity, 20% for five-state, and 12.5% for recall.

| Model                     | Params | parity dense    | parity sparse  | five dense      | five sparse     | recall sparse   |
|---------------------------|--------|-----------------|----------------|-----------------|-----------------|-----------------|
| Elman RNN                 | 76,416 | 100.0 $\pm$ 0.0 | 49.3 $\pm$ 0.5 | 100.0 $\pm$ 0.0 | 100.0 $\pm$ 0.0 | 13.2 $\pm$ 0.3  |
| LSTM                      | 75,648 | 100.0 $\pm$ 0.0 | 49.5 $\pm$ 0.6 | 100.0 $\pm$ 0.0 | 100.0 $\pm$ 0.0 | 13.3 $\pm$ 0.5  |
| GRU                       | 77,280 | 100.0 $\pm$ 0.0 | 49.5 $\pm$ 0.6 | 100.0 $\pm$ 0.0 | 100.0 $\pm$ 0.0 | 100.0 $\pm$ 0.0 |
| Transformer (full causal) | 76,608 | 89.5 $\pm$ 7.7  | 49.5 $\pm$ 0.6 | 73.3 $\pm$ 8.6  | 59.8 $\pm$ 7.3  | 100.0 $\pm$ 0.0 |
| TokenMerge (RLT abl.)     | 82,880 | 100.0 $\pm$ 0.0 | 49.5 $\pm$ 0.6 | 100.0 $\pm$ 0.0 | 100.0 $\pm$ 0.0 | 41.9 $\pm$ 50.3 |
| RLT                       | 75,000 | 100.0 $\pm$ 0.0 | 49.5 $\pm$ 0.6 | 100.0 $\pm$ 0.0 | 98.6 $\pm$ 2.4  | 100.0 $\pm$ 0.0 |
| SCAR                      | 77,896 | 100.0 $\pm$ 0.0 | 49.5 $\pm$ 0.6 | 100.0 $\pm$ 0.0 | 100.0 $\pm$ 0.0 | 100.0 $\pm$ 0.0 |
| SCAR-carrier (no memory)  | 77,280 | 100.0 $\pm$ 0.0 | 49.5 $\pm$ 0.6 | 100.0 $\pm$ 0.0 | 100.0 $\pm$ 0.0 | 100.0 $\pm$ 0.0 |
| SCAR-no-recall            | 78,710 | 100.0 $\pm$ 0.0 | 49.5 $\pm$ 0.6 | 100.0 $\pm$ 0.0 | 100.0 $\pm$ 0.0 | 100.0 $\pm$ 0.0 |

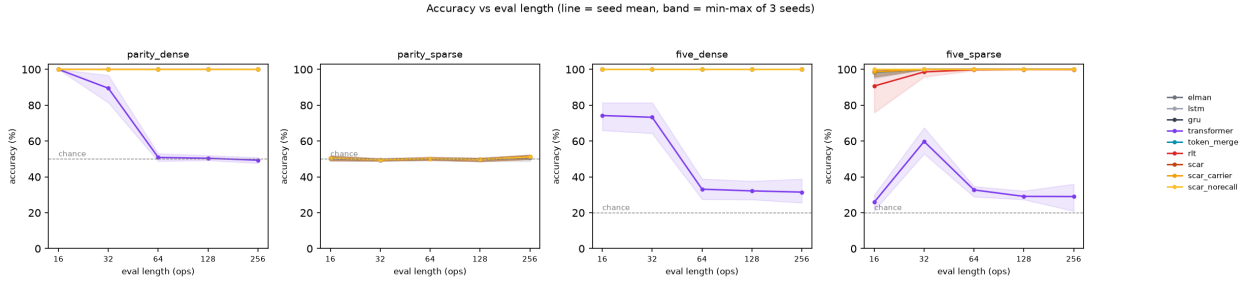


Figure 1: Accuracy versus evaluation length. Lines are seed means and shaded bands span the three-seed minimum and maximum.

## 6 Results

### 6.1 Delayed recall and ablations

At the trained length, the models that clearly beat the 12.5% chance floor are GRU, Transformer (full causal), TokenMerge (RLT abl.), RLT, SCAR, SCAR-carrier (no memory), SCAR-no-recall. The more revealing comparison is extrapolation: at 512 operations, SCAR remains at  $100.0 \pm 0.0\%$ , while the carrier-only and no-recall variants fall to  $56.6 \pm 18.5\%$  and  $30.5 \pm 32.4\%$ , respectively.

The two ablations are parameter-matched to SCAR within roughly one percent: SCAR-carrier widens the GRU carrier to  $d=112$  after removing the memory, while SCAR-no-recall widens  $d$  to 98 while retaining the write path. Both reach  $100.0 \pm 0.0\%$  at the trained recall length, so neither component is required to fit the short training distribution. At 512 operations, however, the full model reaches  $100.0 \pm 0.0\%$  versus  $56.6 \pm 18.5\%$  without memory and  $30.5 \pm 32.4\%$  without the read path. This supports a narrow conclusion: constant-size memory is useful for long-delay extrapolation in this probe, but the experiment does not show that the memory improves every task or every length.

## 7 Compute cost

On this CPU, full SCAR costs  $3.12\times$  a GRU per training step, while the carrier-only ablation costs  $1.04\times$ . SCAR is  $0.40\times$  the measured RLT training-step cost. This falsifies the original  $1\times$ -GRU cost target while supporting a substantial reduction relative to RLT. These are controlled single-machine timings, not hardware-independent complexity claims.

Table 2: Extrapolation accuracy (%) at the longest evaluated length (parity/five: 256 operations; recall: 512 operations). No model was trained at these lengths.

| Model                     | Params | parity dense    | parity sparse  | five dense      | five sparse     | recall sparse   |
|---------------------------|--------|-----------------|----------------|-----------------|-----------------|-----------------|
| Elman RNN                 | 76,416 | 100.0 $\pm$ 0.0 | 50.7 $\pm$ 1.8 | 100.0 $\pm$ 0.0 | 100.0 $\pm$ 0.0 | 12.3 $\pm$ 1.9  |
| LSTM                      | 75,648 | 100.0 $\pm$ 0.0 | 51.1 $\pm$ 1.2 | 100.0 $\pm$ 0.0 | 100.0 $\pm$ 0.0 | 12.6 $\pm$ 1.7  |
| GRU                       | 77,280 | 100.0 $\pm$ 0.0 | 51.1 $\pm$ 1.2 | 100.0 $\pm$ 0.0 | 100.0 $\pm$ 0.0 | 56.6 $\pm$ 18.5 |
| Transformer (full causal) | 76,608 | 49.3 $\pm$ 1.6  | 51.1 $\pm$ 1.2 | 31.5 $\pm$ 6.7  | 29.0 $\pm$ 7.6  | 77.5 $\pm$ 19.7 |
| TokenMerge (RLT abl.)     | 82,880 | 100.0 $\pm$ 0.0 | 51.1 $\pm$ 1.2 | 100.0 $\pm$ 0.0 | 100.0 $\pm$ 0.0 | 8.1 $\pm$ 7.0   |
| RLT                       | 75,000 | 100.0 $\pm$ 0.1 | 51.1 $\pm$ 1.2 | 100.0 $\pm$ 0.0 | 99.9 $\pm$ 0.2  | 100.0 $\pm$ 0.0 |
| SCAR                      | 77,896 | 100.0 $\pm$ 0.0 | 51.1 $\pm$ 1.2 | 100.0 $\pm$ 0.0 | 100.0 $\pm$ 0.0 | 100.0 $\pm$ 0.0 |
| SCAR-carrier (no memory)  | 77,280 | 100.0 $\pm$ 0.0 | 51.1 $\pm$ 1.2 | 100.0 $\pm$ 0.0 | 100.0 $\pm$ 0.0 | 56.6 $\pm$ 18.5 |
| SCAR-no-recall            | 78,710 | 100.0 $\pm$ 0.0 | 51.1 $\pm$ 1.2 | 100.0 $\pm$ 0.0 | 100.0 $\pm$ 0.0 | 30.5 $\pm$ 32.4 |

Table 3: Dedicated serial cost benchmark: 100 timed iterations after 20 warmup, torch threads = 2, batch 64, train length 32+BOS, inference length 128+BOS. Median [min-max] ms.

| Model                     | train ms/step       | $\times$ GRU  | infer ms/batch |
|---------------------------|---------------------|---------------|----------------|
| Elman RNN                 | 23.3 [22.3–27.5]    | 0.75 $\times$ | 29.3           |
| LSTM                      | 29.9 [28.7–47.2]    | 0.96 $\times$ | 36.3           |
| GRU                       | 31.0 [30.0–46.8]    | 1.00 $\times$ | 33.2           |
| Transformer (full causal) | 32.6 [31.6–35.2]    | 1.05 $\times$ | 102.5          |
| TokenMerge (RLT abl.)     | 217.6 [206.5–249.9] | 7.03 $\times$ | 257.1          |
| RLT                       | 243.7 [236.2–280.8] | 7.87 $\times$ | 391.9          |
| SCAR                      | 96.5 [92.6–127.0]   | 3.12 $\times$ | 104.8          |
| SCAR-carrier (no memory)  | 32.2 [31.2–41.1]    | 1.04 $\times$ | 33.7           |
| SCAR-no-recall            | 43.4 [42.4–63.4]    | 1.40 $\times$ | 64.7           |

## 8 Discussion

The clean fixed-length density comparison is descriptive: Under sparse parity supervision, 9 of nine architectures remain at the 50% chance floor (Elman RNN, LSTM, GRU, Transformer (full causal), TokenMerge (RLT abl.), RLT, SCAR, SCAR-carrier (no memory), SCAR-no-recall). Dense supervision makes almost every architecture look successful, whereas sparse parity leaves several at chance. With only three seeds, we report means and sample standard deviations but do not call near-equal cells significance-labeled. The strongest result is therefore not that SCAR universally wins: it is that the parameter-matched carrier-only model can match SCAR on short tasks while losing its long-delay advantage.

The central result is a separation between fitting and extrapolation. At the trained lengths, the SCAR ablations report the same rounded means as SCAR on these small tasks; this is a descriptive comparison, not a significance test. On the 512-operation recall probe, however, the full multi-timescale read path retains perfect performance while the two parameter-matched ablations degrade substantially. This is evidence that the fixed memory can carry a useful long-delay signal, not evidence that every component is needed for every task.

The clean density sweep also changes the story. Dense parity supervision makes almost every architecture look successful, whereas fixed-length sparse parity leaves several at chance. A single architecture ranking under one supervision regime would therefore conflate model capability with the amount and placement of gradient signal. We report this as a descriptive three-seed comparison, not as a statistically tested ranking.

## 9 Limitations

This is a synthetic study with small models, a single parameter scale, three seeds, and short training budgets. The tasks test exact state tracking rather than natural-language modeling, multimodal reasoning, or noisy real-world sequences. The ablations are parameter-matched, but changing width to replace removed modules can itself change optimization and representation capacity. The RLT comparison is a small implementation, not a claim about the behavior of all RLT variants. Timing uses one CPU machine, two PyTorch threads, a parity-sized vocabulary, and an unoptimized research implementation; measured ratios should not be generalized to GPU kernels or production runtimes. Finally, the fixed memory’s success on this recall generator does not establish that learned exponential summaries are sufficient for arbitrary long-context tasks.

## 10 Reproducibility and release

The repository contains the model and task code (“bench.py”), regression tests (“tests/”), all 135 result JSON files, aggregation and chart scripts, the generated tables, and the figures used in this PDF. The sweep itself is run on Kaggle CPU kernels through “kaggle/”; no local training is required. The release command sequence is:

```
python3 -m pytest tests/ -q
python3 aggregate.py
python3 make_charts.py
python3 make_paper.py
(cd paper && tectonic paper.tex)
```

The generated summary includes per-seed values and the curriculum flag so the reported condition cannot silently drift from the trained artifacts.

## References

- [1] J. L. Elman, “Finding structure in time,” *Cognitive Science*, 14(2), 179–211, 1990. doi:10.1207/s15516709cog1402\_1.
- [2] S. Hochreiter and J. Schmidhuber, “Long short-term memory,” *Neural Computation*, 9(8), 1735–1780, 1997.
- [3] K. Cho et al., “Learning phrase representations using RNN encoder–decoder for statistical machine translation,” arXiv:1406.1078, 2014.
- [4] A. Vaswani et al., “Attention is all you need,” *Advances in Neural Information Processing Systems*, 30, 2017. arXiv:1706.03762.
- [5] S. Sukhbaatar, J. Weston, R. Fergus, et al., “End-to-end memory networks,” *Advances in Neural Information Processing Systems*, 28, 2015. arXiv:1503.08895.
- [6] A. Gu, K. Goel, and C. Ré, “Efficiently modeling long sequences with structured state spaces,” in *International Conference on Learning Representations*, 2022. arXiv:2111.00396.
- [7] Z. Dai, Z. Yang, Y. Yang, J. Carbonell, Q. V. Le, and R. Salakhutdinov, “Transformer-XL: Attentive language models beyond a fixed-length context,” *Proceedings of ACL*, 2019. arXiv:1901.02860.
- [8] J. W. Rae, A. Potapenko, S. M. Jayakumar, and T. P. Lillicrap, “Compressive transformers for long-range sequence modelling,” in *International Conference on Learning Representations*, 2020. arXiv:1911.05507.
- [9] S. Arora, S. Eyuboglu, A. Timalsina, I. Johnson, M. Poli, J. Zou, A. Rudra, and C. Ré, “Zoology: Measuring and improving recall in efficient language models,” arXiv:2312.04927, 2023.

- [10] Y. Zhang, J. Feng, and S. Qin, “Recurrent Looped Transformer,” technical report, 2026. <https://github.com/yifanzhang-pro/recurrent-looped-tranformer>.

## Delayed first-token recall: accuracy vs eval length

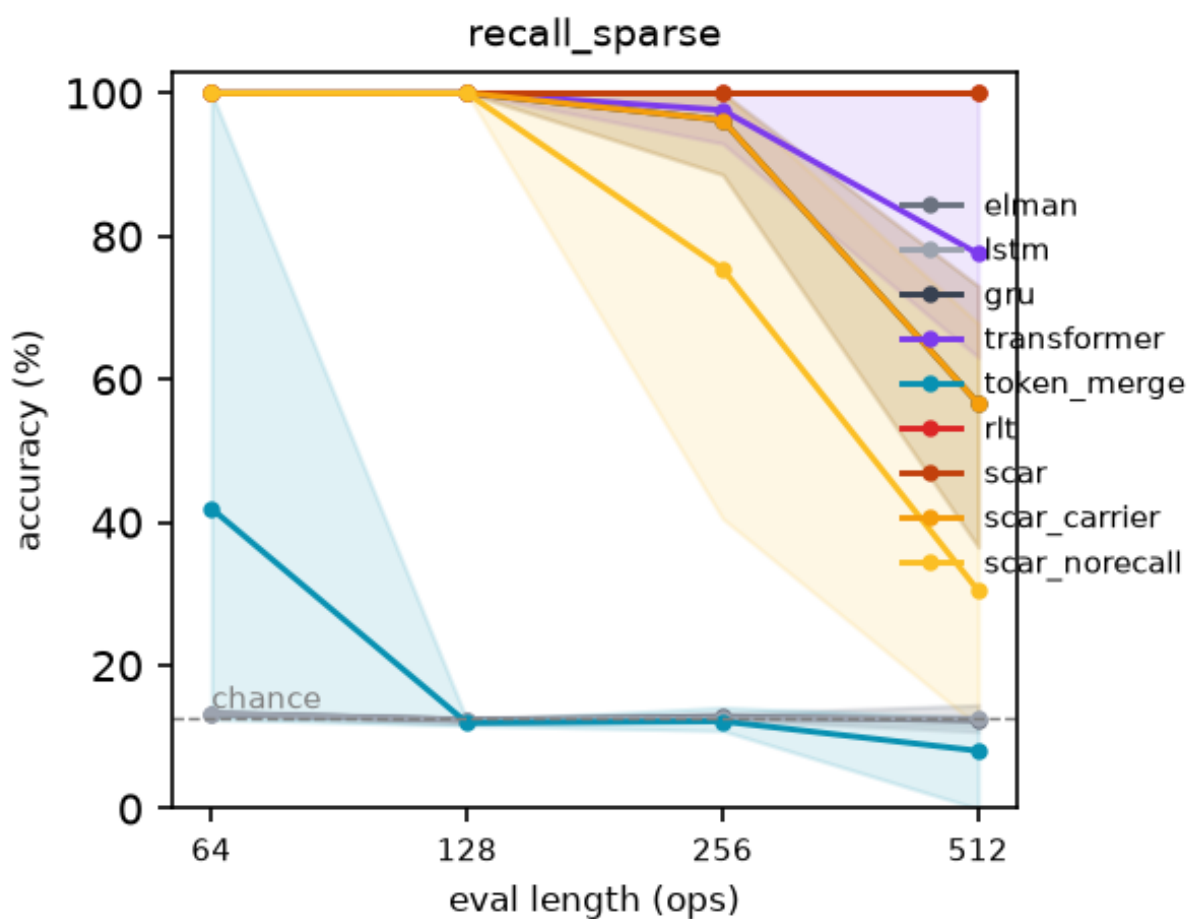


Figure 2: Delayed first-token recall across evaluation lengths; chance is 12.5%.

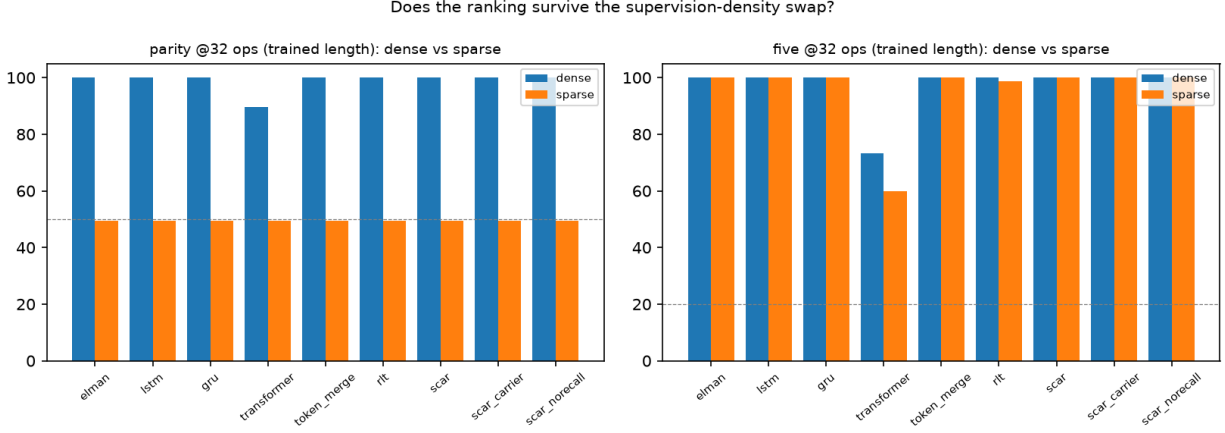


Figure 3: Changing supervision density changes which architectures escape chance on the trained-length parity and five-state tasks.

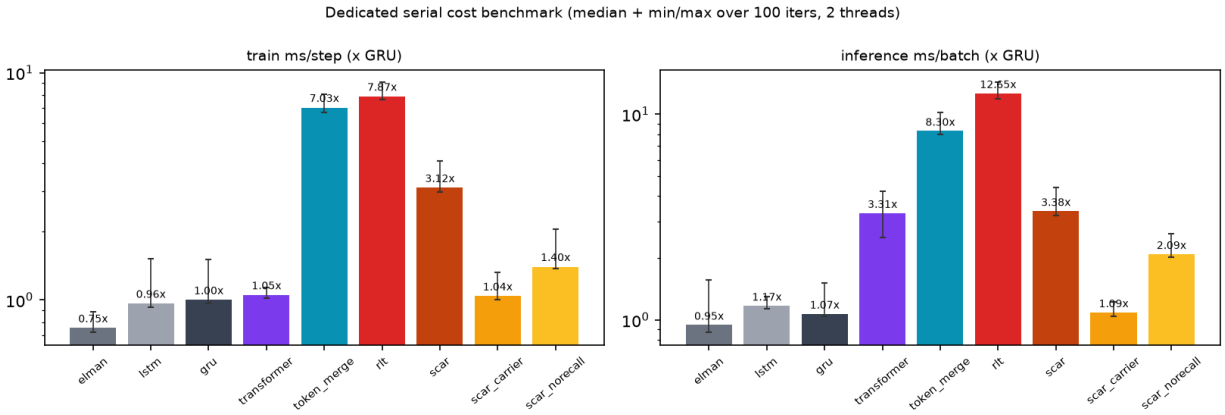


Figure 4: Dedicated serial CPU benchmark. Bars show medians and whiskers show the repeated-iteration minimum and maximum.